Japes and eval-service contract convergence Proposal

Code baseline: Japes `origin/dev` at `dc4e376` (`2.3.3`, 2026-08-05) and eval-service `origin/main` at `edd3cfb` (2026-07-30)

Solution Proposal

Eval-service should use Japes evaluation capabilities, but convergence should happen through versioned contracts and explicit ownership - not through shared database models or by putting eval-service on the Japes runtime.

- **Eval-service remains the feedback and evaluation control plane.** It owns durable feedback, experiment and test-run state, processing and review workflows, MLflow integration, attribution orchestration, learning-candidate state, and reporting.
- **Japes remains the provider-neutral execution and learning toolkit.** It owns reusable scorer implementations, evaluation harnesses, generic trace abstractions, feedback ergonomics for interactive agents, and draft guidance synthesis.
- **Attribution is a platform capability, not a Juno capability.** Juno's current meta-profile analysis becomes one analyzer implementation behind a generic eval-service attribution contract. Japes, Kernel, Juno, Flowable, and future runtimes contribute evidence through adapters.
- **Every accepted feedback reaction starts the same durable reaction pipeline -** persist feedback, resolve source identity, collect evidence, produce a versioned attribution, derive a governed learning candidate, validate it, and only then make approved learning available to runtimes.
- **Shared wire contracts live in a contract-only distribution** published from the Japes repository, provisionally `jazzx-eval-contracts`. It must not import FastAPI, SQLModel, queues, MLflow, Kernel clients, or `JazzXRuntime`.
- **The eval-service API and durable control plane do not run on `JazzXRuntime`.** A dedicated `japes-eval-runner` or agentic attribution-analyzer worker may use `JazzXRuntime` when its responsibility fits the handler model. Those workers remain behind shared contracts and do not own eval-service records or lifecycle. This also helps in gradual adoption

The critical architectural change proposal is that feedback is no longer treated as learning by itself. Feedback is an observation. Attribution determines what the observation is evidence

about. A governed learning candidate proposes what should change. Evaluation and approval determine whether that change may be promoted.

How Japes and eval-service differ today

The systems overlap, but they are not currently expressing the same concepts or lifecycle.

Concern	Japes `origin/dev`	eval-service `origin/main`	Convergence required
Reaction	`Reaction.POSITIVE`, `NEGATIVE`, `NEUTRAL`; turn feedback defaults to neutral	`thumbs_up`, `thumbs_down`; reaction required	Standardize the V1 wire value on `thumbs_up
Rating	Typed `rating: float	None`, described as normalized to` [0,1]`	No typed rating on `FeedbackCreate` or the durable feedback row
Feedback ID	Locally generated prefixed string such as `fb_<uuid>`	Durable UUID generated by eval-service	Use an event UUID for idempotency and an eval-service feedback UUID as the canonical record ID
Entity identity	Optional string subject, case, pack, conversation, message, and decision identifiers	UUID `entity_id` and optional UUID `entity_invocation_id`; other correlations are mostly in metadata	Use a platform UUID for the durable entity; preserve service-native IDs as source-qualified opaque references
Trace linkage	One ambiguous `trace_id`, intended to be resolved through a generic `TraceSource`	A list of IDs that current code assumes are MLflow traces, plus MLflow run/assessment fields	Standardize `TraceRefV1`; Japes stays generic and eval-service maps `kind=mlflow` to its specific persistence and APIs
Feedback persistence	In-process or Japes DB store; intentionally reduced SDK core	PostgreSQL system of record with processing, review, deduplication, embeddings, moderation, and MLflow mirroring	Keep eval-service as the only production source of truth; retain Japes stores for local/offline use
Context correlation	Interactive response carries conversation, message, and one trace ID	Assistant-message ingress probes Assistant traces, then branches into Japes or Kernel-specific metadata	Replace core `context_type` branching with registered evidence-provider adapters

Concern	Japes `origin/dev`	eval-service `origin/main`	Convergence required
Attribution	No evidence-backed attribution contract; `Feedback.to_signal()` immediately maps reaction/text to a priority-tagged `ImprovementSignal`	Rich asynchronous evidence-matrix analysis exists only for `context_type == "juno"`; Japes feedback is correlated but skips Kernel context and does not receive equivalent analysis	Add generic evidence, attribution, and analyzer contracts for every originating service
Learning	Feedback quality/actionable extraction, Curator routing, and `GuidanceAsset` synthesis; synthesis lands in `DRAFT`	Quality/actionable processing and approved-feedback retrieval; attribution output is stored in Juno-shaped metadata	Introduce a shared `LearningCandidateV1` linked to a typed attribution; keep promotion governed
Guidance handoff	`EvalServiceGuidanceStore` fabricates trigger patterns from feedback text, uses `STANDARD` confidence, marks results `DEPLOYED`, and returns flat score `1.0` because the eval API omits those concepts	Similar-feedback read model does not expose a governed guidance contract or result score	Publish an approved-learning read model containing real scope, confidence, provenance, version, and retrieval score
Scoring	`ScorerResult` has name, score, passed, confidence, metrics, metadata, and separate `skipped`	`EvaluationResult` has passed, score, comment, type, metrics, and metadata	Standardize a lossless scorer verdict and a separate aggregate case evaluation
Execution vs. quality	`CaseResult` can contain `passed=False` and an execution error in the same object	`TestCaseRun` has a runtime status but stores evaluation as JSON with another `passed`	Make execution state and quality verdict independent, typed axes

This table is the current-state baseline, not a reason to pick one implementation wholesale. Japes has the stronger provider-neutral SDK shapes and governed draft-guidance primitives. Eval-service has the stronger durable workflow and the richer—currently Juno-specific feedback analysis. The target is to combine those strengths behind stable contracts.

Architectural principles

1. **Feedback acceptance is synchronous and durable; attribution and learning are asynchronous.** An unavailable analyzer must not cause the user's feedback submission to fail.

2. **Feedback, attribution, and learning are separate resources.** Let us not keep extending `Feedback.metadata` into an unversioned aggregate document.
3. **Attribution is evidence-backed.** A cause without supporting evidence and recorded signal checks is a hypothesis, not an attributed cause.
4. **Absence of evidence is represented explicitly.** Providers report unavailable, not-found, denied, and error states; analyzers must not fabricate context.
5. **All services use the same orchestration lifecycle.** Service-specific behavior lives only in evidence providers and optional analyzer implementations.
6. **Canonical platform identity is UUID; source correlation remains opaque.** Do not force external conversation, message, invocation, or trace IDs into UUID merely because a current service happens to use UUIDs.
7. **Positive feedback also participates in learning.** `thumbs_up` can produce a preservation or reinforcement candidate; it is not merely a no-op opposite of a complaint.
8. **Rating changes priority, not truth.** It may influence urgency or sampling, but it cannot substitute for evidence or increase attribution confidence by itself.
9. **No direct feedback-to-production-learning path.** Raw feedback may create a draft candidate, never an approved or deployed asset.
10. **Execution failure and quality failure remain independent.** A successfully executed response can fail evaluation; an infrastructure failure normally has no quality verdict.

Ownership model

Concern	Owner	Extension point or shared boundary
Feedback persistence, idempotency, edits, moderation, and review	eval-service	`FeedbackSubmissionV1`, `FeedbackAcceptedV1`
Attribution job lifecycle and policy	eval-service	`AttributionRequestV1`, analyzer/provider registries
Source evidence retrieval	Originating service	`EvidenceProvider` returning `EvidenceBundleV1`
Generic attribution analysis	Pluggable; initially Juno meta profile behind an adapter	`AttributionAnalyzer` returning `FeedbackAttributionV1`
Learning-candidate lifecycle and governance	eval-service	`LearningCandidateV1`, validation and approval events
Draft guidance synthesis	Japes Curator or another registered builder	Consumes governed learning candidates, not raw feedback

Concern	Owner	Extension point or shared boundary
Approved guidance retrieval and application	Japes/runtime consumers	`ApprovedLearningAssetV1` read model
Experiment and `TestCaseRun` lifecycle	eval-service	Invocation and result events
Japes pack/interactive-agent execution	Assistant/Japes runner, optionally hosted on `JazzXRuntime`	`ExecutionResultV1`
Scorer implementations	Japes	`ScorerVerdictV1`
Scorer selection, thresholds, and aggregation policy	eval-service	Versioned scorer and aggregation references
MLflow assessments and trace persistence	eval-service	MLflow adapter over `TraceRefV1`
Contract schemas and compatibility fixtures	Japes contract distribution; jointly reviewed	JSON Schema plus producer/consumer tests

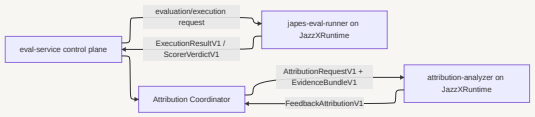
Where `JazzXRuntime` fits

`JazzXRuntime` is a good fit for bounded, handler-shaped execution. It is not the host for the eval-service API, database lifecycle, experiment state machine, feedback system of record, or attribution coordinator.

Two dedicated workers are valid uses:

Worker	Handler responsibility	Input	Output	State it must not own
`japes-eval-runner`	Execute a Japes agent/pack or reusable scorer for one requested unit of work	Versioned evaluation/execution request with entity, testcase-run, security, and trace context	`ExecutionResultV1` and/or `ScorerVerdictV1`	Experiment, `TestCaseRun`, aggregation, feedback, or promotion lifecycle
`attribution-analyzer`	Analyze one normalized feedback-and-evidence snapshot using a Japes agentic implementation	`AttributionRequestV1` plus `EvidenceBundleV1`	`FeedbackAttributionV1`	Feedback persistence, evidence-provider selection, attribution versioning policy, or learning-

Worker	Handler responsibility	Input	Output	State it must not own
				candidate lifecycle



The runtime is therefore an implementation choice behind a worker boundary, not a requirement of the shared contracts. A deterministic scorer or non-Japes analyzer can implement the same boundary without `JazzXRuntime`. We can decide to how to adopt over time..

This separation preserves the operational semantics of both systems for now.

- Eval-service retains durable idempotency, database transactions, outbox processing, domain-specific retry policy, and authoritative state transitions.
- `JazzXRuntime` retains handler startup/shutdown, invocation context, security-context propagation, trace attachment, and bounded queue or HTTP execution.
- Workers scale and deploy independently from the latency-sensitive eval-service API.
- A worker failure is recorded as execution or analysis failure; it cannot take down feedback acceptance or mutate control-plane state directly.
- Upgrading Japes runtime behavior does not require re-hosting or restructuring eval-service's public API.

Using Japes libraries inside an eval-service worker also does not require `JazzXRuntime`. Eval-service may import the contract package, scorer implementations, trace abstractions, feedback-quality utilities, or Curator primitives directly when the work is already managed by its own worker lifecycle.

Contracts to standardize

The shared distribution should expose these contracts. SQLModel rows, Japes internal models, HTTP envelopes, and provider SDK responses remain private implementation details.

Contract	Purpose
`EntityRefV1`	Canonical platform entity identity
`SourceRefV1`	Service-qualified native identity or correlation reference

Contract	Purpose
`TraceRefV1`	Provider-neutral reference to a trace or run
`FeedbackOriginV1`	Which service/runtime/version and tenant context produced the feedback
`FeedbackContextV1`	Typed entity, invocation, message, evaluation, and trace correlation
`FeedbackSubmissionV1`	Immutable feedback fact supplied by a producer
`FeedbackAcceptedV1`	Durable event that triggers processing and attribution
`EvidenceRefV1`	Access-controlled pointer to one evidence item
`SignalCheckV1`	What was checked and whether it was present, absent, denied, or unavailable
`EvidenceBundleV1`	Versioned evidence snapshot assembled for one attribution run
`AttributionRequestV1`	Internal work request connecting feedback, policy, and evidence scope
`CandidateCauseV1`	One evidence-linked causal hypothesis
`FeedbackAttributionV1`	Typed, versioned attribution result independent of the analyzer used
`LearningCandidateV1`	Governed proposal derived from feedback plus attribution
`ApprovedLearningAssetV1`	Stable read model used by Japes and other runtimes
`ScorerVerdictV1`	Lossless result from one scorer
`CaseEvaluationV1`	Aggregate quality decision for a completed case
`ExecutionResultV1`	Runtime outcome, independent of evaluation quality

Identity, origin, and trace contracts

```
class EntityRefV1(BaseModel):
    entity_type: str
    entity_id: UUID


class SourceRefV1(BaseModel):
    service: str                  # "japes", "assistant", "kernel",
"juno", "flowable", ...
    kind: str                     # "conversation", "message", "invoc
ation", "process_instance", ...
    id: str                       # opaque to eval-service
    uri: str | None = None


class TraceRefV1(BaseModel):
    kind: str                     # "mlflow", "otel", "japes", or ano
```

```
ther registered kind
    id: str
    uri: str | None = None
    attributes: dict[str, str] = Field(default_factory=dict)


class FeedbackOriginV1(BaseModel):
    service: str
    runtime: str | None = None
    producer_version: str | None = None
    tenant_id: str | None = None
    workspace_id: str | None = None
    actor_id: str | None = None
    source_url: str | None = None
```

UUID is the correct identity for platform-managed entities and durable workflow records. It provides consistent joins, authorization, indexing, and referential integrity across eval-service. It is not the correct type for every identifier emitted by another system.

For services that do not already know the platform UUID, eval-service should resolve a `SourceRefV1` through an entity-binding table

```
entity_binding(
  service,
  source_kind,
  source_id,
  platform_entity_type,
  platform_entity_id UUID,
  valid_from,
  valid_to
)

CREATE UNIQUE INDEX uq_entity_binding_active
  ON entity_binding(service, source_kind, source_id)
  WHERE valid_to IS NULL
```

The resolved UUID is persisted on feedback. The source reference is retained for provenance. Resolution failure is explicit; eval-service must not invent a UUID or bind a message ID as an agent merely to satisfy the type system. Producers that already know the registered entity UUID should send it directly and avoid the lookup.

`TraceRefV1` deliberately does not make Japes aware of MLflow storage semantics. Eval-service remains specific: it handles `kind == "mlflow"` with its current MLflow APIs and stores other kinds as external provenance. A Japes trace can therefore never be accidentally submitted to an MLflow lookup merely because both are strings.

Feedback contract

```python
class FeedbackSource(str, Enum):
    USER = "user"
    SYSTEM = "system"
    JUDGE = "judge"


class FeedbackReaction(str, Enum):
    THUMBS_UP = "thumbs_up"
    THUMBS_DOWN = "thumbs_down"


class FeedbackContextV1(BaseModel):
    entity: EntityRefV1 | None = None
    source_entity: SourceRefV1 | None = None
    entity_invocation_id: UUID | None = None
    source_refs: list[SourceRefV1] = Field(default_factory=list)
    experiment_id: UUID | None = None
    testcase_id: UUID | None = None
    testcase_run_id: UUID | None = None
    trace_refs: list[TraceRefV1] = Field(default_factory=list)

    @model_validator(mode="after")
    def require_resolvable_entity(self):
        if self.entity is None and self.source_entity is None:
            raise ValueError("entity or source_entity is required")
        return self


class FeedbackSubmissionV1(BaseModel):
    schema_version: Literal["1"] = "1"
    event_id: UUID
    source: FeedbackSource
    source_id: str | None = None
    reaction: FeedbackReaction
    rating: float | None = Field(default=None, ge=0.0, le=1.0)
    text: str | None = None
    category: str | None = None
    origin: FeedbackOriginV1
```

```
    context: FeedbackContextV1
    occurred_at: datetime
    metadata: dict[str, Any] = Field(default_factory=dict)


class FeedbackAcceptedV1(BaseModel):
    schema_version: Literal["1"] = "1"
    event_id: UUID
    feedback_id: UUID
    entity: EntityRefV1
    reaction: FeedbackReaction
    rating: float | None
    origin_service: str
    accepted_at: datetime
```

Rules

- `reaction` is required for new V1 submissions. Japes must stop treating an omitted reaction as neutral.
- Legacy Japes `positive` maps to `thumbs_up`; `negative` maps to `thumbs_down`. Legacy `neutral` remains unclassified and requires explicit producer or reviewer action; it is never silently mapped.
- `rating` is a typed nullable `[0,1]` field throughout eval-service create, read, edit, export, audit, and auto-emission paths. It must not be buried in metadata.
- Reaction is the categorical user judgment. Rating is optional magnitude. V1 does not infer one from the other.
- A producer sends the platform `entity` when it knows the registered UUID. Otherwise it sends `source_entity`, which eval-service must resolve before accepting and publishing the normalized event.
- Eval-service generates `feedback_id`. `event_id` is the producer idempotency key. Replaying the same event returns the same feedback and does not duplicate MLflow assessments, outbox events, attribution runs, or learning candidates.
- `FeedbackSubmissionV1` is the immutable submitted fact. Later edits are audited revisions; derived attribution is not written back into this contract.

Evidence and attribution contracts

Attribution needs both evidence references and an audit of what was checked. Merely returning a summary, confidence number, and blob path is not sufficient for cross-service

learning.

```python
class EvidenceAvailability(str, Enum):
    PRESENT = "present"
    NOT_FOUND = "not_found"
    UNAVAILABLE = "unavailable"
    ACCESS_DENIED = "access_denied"
    ERROR = "error"


class EvidenceRefV1(BaseModel):
    evidence_id: UUID
    source: SourceRefV1
    evidence_type: str             # message, tool_call, prompt, guid
ance, process_event, ...
    captured_at: datetime
    content_hash: str | None = None
    summary: str | None = None
    sensitivity: str = "internal"


class SignalCheckV1(BaseModel):
    provider: str
    signal: str                    # "tool_call", "guidance_refs", "m
odel_output", ...
    availability: EvidenceAvailability
    evidence_ids: list[UUID] = Field(default_factory=list)
    detail: str | None = None


class EvidenceBundleV1(BaseModel):
    evidence_bundle_id: UUID
    feedback_id: UUID
    snapshot_version: int
    providers: list[str]
    evidence: list[EvidenceRefV1]
    signal_checks: list[SignalCheckV1]
    snapshot_hash: str
    collected_at: datetime
```

```python
class AttributionStrength(str, Enum):
    CONFIRMED = "confirmed"
    LIKELY = "likely"
    POSSIBLE = "possible"
    UNATTRIBUTED = "unattributed"


class CandidateCauseV1(BaseModel):
    component: SourceRefV1
    cause_type: str
    description: str
    confidence: float = Field(ge=0.0, le=1.0)
    supporting_evidence_ids: list[UUID]
    contradicting_evidence_ids: list[UUID] = Field(default_factory=li
st)


class FeedbackAttributionV1(BaseModel):
    schema_version: Literal["1"] = "1"
    attribution_id: UUID
    feedback_id: UUID
    attribution_version: int
    status: Literal["completed", "insufficient_evidence"]
    strength: AttributionStrength
    primary_component: SourceRefV1 | None = None
    candidate_causes: list[CandidateCauseV1] = Field(default_factory=
list)
    observed_behavior: str
    summary: str
    recommended_action: str | None = None
    recommended_next_signal: str | None = None
    evidence_bundle_id: UUID
    analyzer: SourceRefV1
    analyzer_version: str
    confidence: float = Field(ge=0.0, le=1.0)
    created_at: datetime
```

The full evidence payload should remain in its source system or an access-controlled immutable artifact store. Eval-service persists queryable attribution fields and evidence

pointers, not copies of every prompt, tool response, document, or process variable. Access to an attribution must be no broader than access to its evidence.

`confidence` is the analyzer's calibrated belief given the collected evidence. `strength` is the policy-readable classification. Neither is a quality score for the original response, and neither is increased merely because feedback was strongly negative.

Feedback-to-learning contract

Attribution is critical because it is the boundary between a reaction and useful learning information. The reaction handler must not call `Feedback.to_signal()` and treat a complaint as a known change target. It must first discover what component or behavior the feedback actually concerns.

```
class LearningIntent(str, Enum):
    CORRECT = "correct"
    PRESERVE = "preserve"
    INVESTIGATE = "investigate"


class LearningChangeType(str, Enum):
    PROMPT = "prompt"
    GUIDANCE = "guidance"
    SKILL = "skill"
    TOOL = "tool"
    POLICY = "policy"
    MODEL = "model"
    PROCESS = "process"
    DATA = "data"
    UNKNOWN = "unknown"


class LearningCandidateV1(BaseModel):
    schema_version: Literal["1"] = "1"
    learning_candidate_id: UUID
    feedback_id: UUID
    attribution_id: UUID
    attribution_version: int
    intent: LearningIntent
    target: SourceRefV1 | None
    change_type: LearningChangeType
    title: str
```

```
    proposed_change: str
    rationale: str
    scope: dict[str, str] = Field(default_factory=dict)
    confidence: float = Field(ge=0.0, le=1.0)
    evidence_ids: list[UUID]
    regression_risks: list[str] = Field(default_factory=list)
    evaluation_plan: str
    status: Literal[
        "draft", "needs_evidence", "in_review", "approved_for_evaluat
ion",
        "evaluating", "promoted", "rejected", "superseded"
    ]
    created_at: datetime
```

Reaction policy:

Reaction	Attribution result	Learning intent
`thumbs_down`	Strong and actionable	`CORRECT` the attributed behavior/component
`thumbs_down`	Weak or insufficient	`INVESTIGATE`; request the next useful signal rather than guessing
`thumbs_up`	Strong and reusable	`PRESERVE`; capture successful behavior or guidance with an anti-overgeneralization scope
`thumbs_up`	Weak, generic, or accidental	Retain for analytics; no candidate required

Japes `ImprovementSignal` can remain an internal Curator input, but the production conversion becomes:

```
LearningCandidateV1 (reviewed/eligible)
    -> ImprovementSignal with feedback_id + attribution_id + evidence provenance
    -> Curator synthesis
    -> GuidanceAsset(status=DRAFT)
    -> evaluation and SME approval
    -> deploy
```

For service-backed feedback, direct `Feedback.to_signal()` should be deprecated or explicitly labeled local/offline only. The current behavior—negative means high priority, everything else low—does not know the affected component, evidence, confidence, safe scope, or whether the feedback is actionable. Those are precisely the facts attribution must supply.

Generic reaction-to-attribution-to-learning flow

1. Accept and publish

Eval-service validates the contract, resolves the entity binding if necessary, persists feedback, and writes `FeedbackAcceptedV1` to a transactional outbox in the same commit. The API returns after durable acceptance. It does not synchronously call Juno, Assistant, Kernel, or Japes.

This changes the current best-effort dispatch from an API/processor special case into a resumable workflow. Feedback remains accepted even when attribution infrastructure is down.

2. Select evidence providers

The Attribution Coordinator selects providers from typed origin and source references. Core orchestration contains no `if context_type == "juno"` or `if context_type == "japes"` logic.

```
class EvidenceProvider(Protocol):
    provider_name: str
    supported_services: set[str]

    async def collect(
        self,
        request: AttributionRequestV1,
        security_context: SecurityContext,
    ) -> EvidenceProviderResultV1: ...
```

Providers return partial evidence plus `SignalCheckV1` records. They do not decide the cause and do not write learning. Adding a future service requires registering a provider and source-reference kinds; it must not require an eval-service schema fork.

3. Analyze generically

An `AttributionAnalyzer` consumes only the normalized feedback and `EvidenceBundleV1`:

```
class AttributionAnalyzer(Protocol):
    analyzer_name: str
    analyzer_version: str

    async def analyze(
```

```
        self,
        feedback: FeedbackReadV1,
        evidence: EvidenceBundleV1,
    ) -> FeedbackAttributionV1: ...
```

The current Juno meta profile can be the first implementation, wrapped by an adapter that converts its evidence-matrix output into `FeedbackAttributionV1`. Juno's `job_id`, profile, webhook envelope, and blob format remain implementation details. The eval-service public resource is `attribution`, not `juno_analysis`.

This separation permits a later Japes-native analyzer, a deterministic analyzer for known error signatures, or a domain analyzer for Flowable without changing feedback persistence or learning consumers.

4. Build learning information

The Learning Candidate Builder combines the reaction, typed rating, attribution, feedback quality/actionability assessment, and governance policy.

- It creates correction candidates from supported negative feedback.
- It creates preservation candidates from specific positive feedback when the behavior is demonstrably reusable.
- It creates investigation candidates when the likely next signal is known but current evidence is insufficient.
- It can decline to create a candidate for vague, duplicated, moderated, or unsupported feedback.
- It records an evaluation plan before a change can advance.

This builder is a reaction handler in the architectural sense, but it is not a synchronous UI callback. It is an idempotent, durable consumer of completed attribution.

5. Govern and measure learning

Candidates move through review and evaluation before promotion. Japes Curator may deduplicate and synthesize a candidate batch into `GuidanceAsset(status=DRAFT)`; existing `GuidanceLifecycle.approve()` and `.deploy()` retain their governance role. Curator never approves or deploys its own proposal.

Approved assets need an explicit read contract so Japes no longer has to invent trigger patterns, confidence, deployment state, or relevance scores from an eval-service feedback row. The read model should include asset ID and version, target/scope, content, trigger patterns, confidence, provenance (`feedback_id`, `attribution_id`, evidence IDs), lifecycle status, and retrieval score.

Learning effectiveness should be measured through the full provenance chain:

```
feedback -> attribution version -> learning candidate -> asset version
         -> evaluation run -> deployment -> downstream outcome
```

That chain supports Japes' existing guidance-effectiveness and reusable-asset-growth concepts without pretending that every feedback row became learning.

Attribution lifecycle and failure semantics

Job execution state and attribution strength are separate axes, just as runtime state and quality verdict are separate.

```python
class AttributionJobStatus(str, Enum):
    PENDING = "pending"
    COLLECTING_EVIDENCE = "collecting_evidence"
    ANALYZING = "analyzing"
    COMPLETED = "completed"
    INSUFFICIENT_EVIDENCE = "insufficient_evidence"
    FAILED = "failed"
    STALE = "stale"
```

Job status	Attribution strength	Meaning
`COMPLETED`	`CONFIRMED`, `LIKELY`, or `POSSIBLE`	Analyzer completed and returned an evidence-linked result
`INSUFFICIENT_EVIDENCE`	`UNATTRIBUTED`	Workflow worked, but the available evidence cannot support a cause
`FAILED`	absent	Provider/analyzer infrastructure failed; retry policy applies
`STALE`	prior value retained	Feedback, binding, analyzer policy, or relevant evidence changed; a new version is required

Required behavior:

- Provider timeout or denial is recorded in signal checks; it is not rewritten as “no problem found.”
- Transient infrastructure failure retries with backoff and a dead-letter/audit path.
- Permanent absence can complete as `INSUFFICIENT_EVIDENCE` with `recommended_next_signal`.
- A feedback edit, entity rebinding, or material new evidence creates a new evidence snapshot and attribution version. Prior versions remain immutable.

- Attribution runs are idempotent on `(feedback_id, analyzer_name, analyzer_version, evidence_snapshot_hash)`.
- Callback delivery is idempotent on provider job ID and attribution run ID.
- A new attribution version supersedes, but does not silently mutate, learning candidates based on an older version.

Scoring contract

The shared result must be lossless without carrying contradictory `passed` and `skipped` booleans.

```
class QualityVerdict(str, Enum):
    PASS = "pass"
    FAIL = "fail"
    SKIP = "skip"


class ScorerRefV1(BaseModel):
    name: str
    evaluator_type: str
    version: str | None = None


class ScorerVerdictV1(BaseModel):
    scorer: ScorerRefV1
    verdict: QualityVerdict
    score: float | None = Field(default=None, ge=0.0, le=1.0)
    confidence: float | None = Field(default=None, ge=0.0, le=1.0)
    threshold: float | None = Field(default=None, ge=0.0, le=1.0)
    comment: str | None = None
    metrics: dict[str, float] = Field(default_factory=dict)
    metadata: dict[str, Any] = Field(default_factory=dict)


class CaseEvaluationV1(BaseModel):
    testcase_id: UUID
    testcase_run_id: UUID
    verdict: QualityVerdict
    score: float | None
    aggregation_policy: str
```

```
    aggregation_policy_version: str
    scorer_verdicts: list[ScorerVerdictV1]
```

Rules:

- `ScorerVerdictV1` is one scorer result; `CaseEvaluationV1` is the aggregate quality judgment.
- Japes owns reusable scorer implementations. Eval-service owns scorer selection, thresholds, required/optional policy, and aggregation.
- Japes `skipped=True` maps to `SKIP`; otherwise `passed` maps to `PASS` or `FAIL`.
- Eval-service `passed` maps to `PASS` or `FAIL` until its internal type is migrated.
- `aggregation_policy_version` makes a result reproducible.
- Metric names are namespaced when semantics differ.

Execution state versus quality verdict

`passed=False` exists today in Japes `ScorerResult`, Japes `CaseResult`, and eval-service `EvaluationResult`. It is valid for a quality check to fail. It is not valid to infer an infrastructure failure from that result.

```
class ExecutionStatus(str, Enum):
    PENDING = "pending"
    RUNNING = "running"
    COMPLETED = "completed"
    FAILED = "failed"
    TIMEOUT = "timeout"
    CANCELLED = "cancelled"


class ExecutionResultV1(BaseModel):
    testcase_run_id: UUID
    external_invocation: SourceRefV1 | None
    status: ExecutionStatus
    output: dict[str, Any] | None
    failure: StructuredFailure | None
    trace_refs: list[TraceRefV1] = Field(default_factory=list)
```

Execution state	Quality verdict	Meaning
`COMPLETED`	`PASS`	Invocation completed and met quality policy

Execution state	Quality verdict	Meaning
`COMPLETED`	`FAIL`	Invocation completed but its result was unacceptable
`COMPLETED`	`SKIP`	Invocation completed but a scorer or policy did not apply
`FAILED`, `TIMEOUT`, `CANCELLED`	absent by default	Invocation did not produce a normally scorable result

A future policy may explicitly score partial output after failure or timeout, but it must record that policy. It is not the default semantic.

API and SDK changes

Japes

- Publish the contract-only distribution and JSON Schema fixtures.
- Add adapters between current `Feedback`, `ScorerResult`, `CaseResult`, and the V1 contracts.
- Make `thumbs_up | thumbs_down` explicit at the shared boundary; stop defaulting new interactive-turn feedback to neutral.
- Replace the canonical one-string trace handoff with `TraceRefV1[]` while preserving internal compatibility adapters.
- Add an authenticated, idempotent `EvalServiceFeedbackSink`; keep in-process/DB feedback stores for offline and test use.
- Add provenance-aware conversion from `LearningCandidateV1` to `ImprovementSignal`; restrict direct raw feedback conversion to local/offline flows.
- Extend `GuidanceProvenance` to carry `attribution_id`, evidence IDs, and learning-candidate ID/version.
- Consume `ApprovedLearningAssetV1` so the guidance adapter no longer fabricates lifecycle, triggers, confidence, or retrieval score.

eval-service

- Add typed `rating` to persistence and every feedback surface.
- Accept `FeedbackSubmissionV1`, generate the feedback UUID, and deduplicate on `event_id`.
- Add entity-binding resolution for producers that only know service-native identity.
- Persist generic source and trace references; keep MLflow-specific columns and behavior behind a provider adapter.

- Add transactionally published `FeedbackAcceptedV1` and a resumable Attribution Coordinator.
- Replace Juno-only dispatch and callback storage with generic attribution job/result resources; retain the Juno client as an adapter.
- Register Japes/Assistant, Kernel, Juno, Flowable, and evaluation evidence providers incrementally.
- Persist typed `FeedbackAttributionV1` and `LearningCandidateV1` resources rather than extending feedback metadata.
- Publish an approved-learning read endpoint with real provenance, confidence, scope, version, and relevance score.
- Persist `CaseEvaluationV1` independently from `TestCaseRun.status`.

Originating services

- Emit typed origin, source references, registered entity UUID when known, and every available trace reference.
- Implement an authenticated evidence provider with least-privilege access and stable not-found/unavailable semantics.
- Return evidence pointers and bounded summaries instead of copying unrestricted payloads.
- Preserve tenant/workspace and actor authorization through evidence collection.
- Do not embed service-specific fields directly into the generic feedback or attribution schemas.

Acceptance criteria

1. Japes, Juno, Kernel, Flowable, and an evaluation run can submit the same `FeedbackSubmissionV1` shape without adding service-specific top-level fields.
2. Retrying a submission creates one feedback row, one accepted event, one analysis per evidence/analyzer version, and no duplicate MLflow assessment or learning candidate.
3. Eval-service preserves rating as a typed `[0,1]` field in create, read, edit, audit, export, and auto-emission flows.
4. A Japes-origin feedback item receives a typed attribution through the same coordinator and lifecycle as a Juno-origin item.
5. Adding a new origin service requires an evidence-provider registration, not changes to core orchestration branches or database columns.

6. Every completed attribution exposes candidate causes, confidence, supporting/contradicting evidence IDs, signal checks, analyzer version, and evidence snapshot version.
7. Unavailable evidence produces an explicit signal-check state and cannot be presented as a confirmed cause.
8. A feedback edit or new evidence produces a new immutable attribution version and marks dependent candidates for review.
9. Negative feedback with insufficient evidence creates an investigation candidate or no candidate—not a guessed corrective instruction.
10. Specific, supported positive feedback can create a preservation candidate with bounded scope and an evaluation plan.
11. No learning candidate can become a deployed guidance asset without evaluation and approval.
12. Japes retrieves approved learning with real scope, provenance, confidence, triggers, version, and relevance score.
13. A Japes scorer round-trips name, type, version, verdict, score, confidence, threshold, metrics, comment, and metadata without loss.
14. `TestCaseRun.status=completed` with `CaseEvaluationV1.verdict=fail` is valid; runtime failure does not automatically create a quality failure.
15. Shared contracts contain no MLflow-specific field, while eval-service continues its current MLflow behavior for `TraceRefV1(kind="mlflow")`.
16. Importing shared contracts initializes no Japes runtime infrastructure.

Security, tenancy, and observability requirements

- Evidence retrieval propagates the original workspace/tenant boundary and a service identity authorized for the specific evidence class.
- Evidence references are access controlled at read time; possessing a feedback UUID does not grant access to prompts, tool payloads, documents, or process variables.
- Sensitive content is minimized, classified, and retained under source-system policy. Hashes and pointers are preferred over duplication.
- Analyzer prompts and artifacts are auditable and versioned. Prompt injection in source evidence is treated as untrusted data.
- Each stage logs `feedback_id`, `attribution_id`, `evidence_bundle_id`, and `learning_candidate_id` where applicable.

- Metrics cover feedback-to-attribution latency, provider coverage, evidence availability, analyzer failures, attribution-strength distribution, candidate conversion, evaluation pass rate, promotion rate, rollback rate, and downstream effectiveness.
- Provider/analyzer execution traces are distinct from the trace being analyzed; both are represented by typed references so observability cannot be mistaken for evidence.

Current code anchors behind this proposal

These anchors explain which behavior is being retained, adapted, or retired. They should be revalidated when implementation begins if either reference has advanced.

Japes

- `jazzx_sdk/evaluation/feedback.py`: current `Reaction` vocabulary, neutral default, typed rating, single trace ID, `from_case_result()`, and direct `to_signal()` conversion.
- `jazzx_sdk/evaluation/feedback_quality.py`: reusable feedback quality, actionable-item, and structured-guidance operations; intentionally excludes persistence, moderation, embeddings, and MLflow.
- `jazzx_sdk/evaluation/scorers.py`: richer current `ScorerResult`, including name, confidence, and skipped.
- `jazzx_sdk/evaluation/harness/results.py`: current `CaseResult` quality flag and execution error in one result shape.
- `jazzx_sdk/modes/evolve/curator.py`: signal routing and LLM synthesis; synthesized `GuidanceAsset` remains `DRAFT` and is not automatically approved or deployed.
- `jazzx_sdk/fabric/guidance/eval_service_store.py`: read-only adaptation that currently supplies fallback triggers, confidence, lifecycle state, and a flat relevance score because the eval-service response does not expose them.

eval-service

- `app/feedback/models.py`: UUID entity identity, up/down reaction, MLflow-oriented trace fields, durable processing/review fields, and the current absence of typed rating.
- `app/feedback/api.py::_correlate_assistant_message`: current Assistant lookup and hard branch between Japes-backed and Kernel-backed messages.
- `app/feedback/processor.py::_maybe_dispatch_juno_analysis`: attribution dispatch currently restricted to Juno feedback.
- `app/feedback/processor.py::_is_japes` and `_fetch_context`: Japes-correlated feedback currently skips Kernel context and has no replacement evidence fetch.

- `app/feedback/juno_client.py::dispatch_feedback_analysis`: current Juno-specific meta-profile job, service identity, interaction payload, and webhook.
- `app/feedback/api.py::juno_analysis_callback`: current callback lifts a small field set into `metadata.analysis`; the proposed typed attribution resource replaces this as the canonical result.
- `app/evals/evaluators.py` and `app/evals/models.py`: current quality `passed` result alongside the separate `TestCaseRun` execution lifecycle.

Explicit non-goals

- Replacing MLflow in eval-service.
- Hosting the eval-service API or durable control plane on `JazzXRuntime`; dedicated execution or analyzer workers are explicitly allowed.
- Sharing eval-service SQLModel persistence classes with Japes.
- Making Japes local feedback storage a second production source of truth.
- Requiring Juno as the permanent attribution engine.
- Copying all source-system evidence into eval-service.
- Forcing external conversation, message, invocation, process, or trace IDs into UUID.
- Automatically converting raw feedback into production guidance.
- Treating every thumbs-up or thumbs-down as sufficiently specific to produce learning.